FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

LABEXER1

Johan Jb A. Bergonia	Sir Hadji Tejuco
Name of Student	**Name of Professor**
October 16, 2026	October 16, 2026
Data Performed	**Date Submitted**

I. Test Cases

Test 1: Addition

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: A

Enter first number: 11
Enter second number: 22
The sum is 33

Do you want to try again (Y/N) Y
---------------------------------
```

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: A

Enter first number: 33
Enter second number: 44
The sum is 77

Do you want to try again (Y/N) Y
---------------------------------
```

```
----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: A

Enter first number: 55
Enter second number: 66
The sum is 121

Do you want to try again (Y/N) N
@Jabergonia →/workspaces/JBRepo (main) $
```

Test 2: Subtraction

```
----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: S

Enter first number: 22
Enter second number: 11
The difference is 11

Do you want to try again (Y/N) Y
----------------------------------
```

```
----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: S

Enter first number: 33
Enter second number: 44
The difference is -11

Do you want to try again (Y/N) Y
----------------------------------
```

```
----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: S

Enter first number: 55
Enter second number: 0
The difference is 55

Do you want to try again (Y/N) N
@Jabergonia →/workspaces/JBRepo (main) $
```

Test 3: Multiplication

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: M

Enter first number: 11
Enter second number: 22
The product is 242

Do you want to try again (Y/N) Y
---------------------------------
```

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: M

Enter first number: 33
Enter second number: 44
The product is 1452

Do you want to try again (Y/N) Y
---------------------------------
```

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: M

Enter first number: 55
Enter second number: 0
The product is 0

Do you want to try again (Y/N) N
@Jabergonia →/workspaces/JBRepo (main) $
```

Test 4: Division

```
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: D

Enter first number: 22
Enter second number: 11
The quotient is 2, remainder is 0

Do you want to try again (Y/N) Y
---------------------------------
```

```
 ----------------------------------
 MENU
 [A]dd
 [S]ubtract
 [M]ultiply
 [D]ivide
 ----------------------------------
 Enter your choice: D

 Enter first number: 44
 Enter second number: 13
 The quotient is 3, remainder is 5

 Do you want to try again (Y/N) Y
 ----------------------------------
```

```
 ----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: D

Enter first number: 55
Enter second number: 0
Error: cannot divide by zero

Do you want to try again (Y/N) N
@Jabergonia →/workspaces/JBRepo (main) $
```

II. Compilation Process

Test 1: Addition

```
Problems  Output  Debug Console  Terminal  Ports

@Jabergonia ➜/workspaces/JBRepo (main) $ nasm -f elf32 LABEXER1.asm -o LABEXER1.o
@Jabergonia ➜/workspaces/JBRepo (main) $ ld -m elf_i386 LABEXER1.o -o LABEXER1
@Jabergonia ➜/workspaces/JBRepo (main) $ ./LABEXER1
----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: A
```

Test 2: Subtraction

```
Problems  Output  Debug Console  Terminal  Ports

@Jabergonia ➜/workspaces/JBRepo (main) $ nasm -f elf32 LABEXER1.asm -o LABEXER1.o
@Jabergonia ➜/workspaces/JBRepo (main) $ ld -m elf_i386 LABEXER1.o -o LABEXER1
@Jabergonia ➜/workspaces/JBRepo (main) $ ./LABEXER1
----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: S
```

Test 3: Multiplication

```
Problems  Output  Debug Console  Terminal  Ports

@Jabergonia ➜/workspaces/JBRepo (main) $ nasm -f elf32 LABEXER1.asm -o LABEXER1.o
@Jabergonia ➜/workspaces/JBRepo (main) $ ld -m elf_i386 LABEXER1.o -o LABEXER1
@Jabergonia ➜/workspaces/JBRepo (main) $ ./LABEXER1
----------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
----------------------------------
Enter your choice: M
```

Test 4: Division

```
Problems  Output  Debug Console  Terminal  Ports

@Jabergonia →/workspaces/JBRepo (main) $ nasm -f elf32 LABEXER1.asm -o LABEXER1.o
@Jabergonia →/workspaces/JBRepo (main) $ ld -m elf_i386 LABEXER1.o -o LABEXER1
@Jabergonia →/workspaces/JBRepo (main) $ ./LABEXER1
---------------------------------
MENU
[A]dd
[S]ubtract
[M]ultiply
[D]ivide
---------------------------------
Enter your choice: D
```

III. Explanation of the Program

Introduction

This calculator program uses a menu-driven approach to allow users to perform addition, subtraction, multiplication, and division on two numbers. It relies on Linux system calls for displaying text, receiving input, and exiting the program. The overall flow consists of showing a menu, accepting user input, performing the selected operation, displaying the result, and asking whether the user wants to continue.

1. Print Macro

```
%macro print 2
...
%endmacro
```

This macro simplifies output operations by wrapping the sys_write system call into a reusable command. It allows the program to print any message by providing only its address and length.

2. Data Section (Messages and Menu)

```
section .data
...
```

This section stores all fixed text used by the program, such as the menu, prompts, result labels, and error messages. These values remain unchanged throughout program execution.

3. BSS Section (Variables)

```
section .bss
...
```

This section reserves memory for user input, temporary buffers, numbers, quotient, and remainder values. The variables are created here because their contents will change while the program runs.

4. Program Entry and Menu Display

```
_start:
main_menu:
print clr, clr_len
print menu, menu_len
```

This block serves as the starting point of the program. It clears the screen and displays the calculator menu whenever a new calculation cycle begins.

5. Menu Choice Processing

```
call read_char
cmp al, 'A'
...
```

This block reads the user's menu selection and compares it with the valid options. Based on the entered letter, execution jumps to the corresponding arithmetic operation.

6. Invalid Choice Handling

```
print msg_bad, msg_bad_len
jmp ask_again
```

This block executes when the user enters an invalid menu option. It displays an error message and proceeds to the retry prompt.

7. Addition Operation

```
do_add:
...
add eax, [num2]
```

This block obtains two numbers from the user and adds them together. The resulting sum is then passed to the output routine for display.

8. Subtraction Operation

```
do_sub:
...
sub eax, [num2]
```

This block subtracts the second number from the first number. It also supports negative results, which will be handled correctly when displayed.

9. Multiplication Operation

```
do_mul:
...
imul eax, [num2]
```

This block multiplies the two user-supplied numbers using signed multiplication. The product is stored in a register and later shown to the user.

10. Division Operation

do_div:
...
div dword [num2]
This block first checks whether the divisor is zero before performing the division. If valid, it calculates both the quotient and the remainder of the operation.

11. Divide-by-Zero Error

div_zero:
print msg_zero, msg_zero_len
This block prevents an invalid division operation from occurring. It informs the user that division by zero is not allowed.

12. Retry and Program Exit

ask_again:
...
exit_program:
This block asks whether the user wants to perform another calculation. Entering Y returns to the menu, while any other input terminates the program.

13. Number Input Procedure

get_numbers:
...
This procedure prompts the user to enter the first and second numbers. Both values are stored in memory for later use by the selected arithmetic operation.

14. Character and Number Reading Procedures

read_char:
...

read_number:
...
These procedures handle all keyboard input for the program. read_char obtains a single menu character, while read_number converts numeric text entered by the user into an integer value.

15. Number Printing Procedure

print_number:
...

This procedure converts a numeric value into its ASCII representation so it can be displayed on the screen. It also handles negative numbers and ensures the digits appear in the correct order when printed.

IV. Source Code

```
%macro print 2
    mov eax, 4
    mov ebx, 1
    mov ecx, %1
    mov edx, %2
    int 0x80
%endmacro

section .data
    clr         db 27, "[2J", 27, "[H"
    clr_len     equ $ - clr

    menu        db "----------------------------------", 10
                db "MENU", 10
                db "[A]dd", 10
                db "[S]ubtract", 10
                db "[M]ultiply", 10
                db "[D]ivide", 10
                db "----------------------------------", 10
                db "Enter your choice: "
    menu_len    equ $ - menu

    msg_first   db 10, "Enter first number: "
    msg_first_len equ $ - msg_first
    msg_second  db "Enter second number: "
    msg_second_len equ $ - msg_second

    msg_sum     db "The sum is "
    msg_sum_len equ $ - msg_sum
    msg_diff    db "The difference is "
    msg_diff_len equ $ - msg_diff
    msg_prod    db "The product is "
    msg_prod_len equ $ - msg_prod
    msg_quot    db "The quotient is "
    msg_quot_len equ $ - msg_quot
    msg_rem     db ", remainder is "
    msg_rem_len equ $ - msg_rem
```

```
    msg_zero    db "Error: cannot divide by zero", 10
    msg_zero_len equ $ - msg_zero
    msg_bad     db 10, "Invalid choice. Please pick A, S, M or D.", 10
    msg_bad_len equ $ - msg_bad

    msg_again   db 10, "Do you want to try again (Y/N) "
    msg_again_len equ $ - msg_again

    newline     db 10

section .bss
    buf     resb 16
    numbuf  resb 12
    num1    resd 1
    num2    resd 1
    quot    resd 1
    rem     resd 1

section .text
    global _start

_start:
main_menu:
    print clr, clr_len
    print menu, menu_len

    call read_char
    cmp al, 'A'
    je  do_add
    cmp al, 'S'
    je  do_sub
    cmp al, 'M'
    je  do_mul
    cmp al, 'D'
    je  do_div

    print msg_bad, msg_bad_len
    jmp ask_again

do_add:
    call get_numbers
    mov eax, [num1]
    add eax, [num2]
```

```
    push eax
    print msg_sum, msg_sum_len
    pop eax
    call print_number
    print newline, 1
    jmp ask_again

do_sub:
    call get_numbers
    mov eax, [num1]
    sub eax, [num2]
    push eax
    print msg_diff, msg_diff_len
    pop eax
    call print_number
    print newline, 1
    jmp ask_again

do_mul:
    call get_numbers
    mov eax, [num1]
    imul eax, [num2]
    push eax
    print msg_prod, msg_prod_len
    pop eax
    call print_number
    print newline, 1
    jmp ask_again

do_div:
    call get_numbers
    cmp dword [num2], 0
    je  div_zero
    mov eax, [num1]
    xor edx, edx
    div dword [num2]
    mov [quot], eax
    mov [rem], edx
    print msg_quot, msg_quot_len
    mov eax, [quot]
    call print_number
    print msg_rem, msg_rem_len
    mov eax, [rem]
```

```
    call print_number
    print newline, 1
    jmp ask_again

div_zero:
    print msg_zero, msg_zero_len

ask_again:
    print msg_again, msg_again_len
    call read_char
    cmp al, 'Y'
    je  main_menu

exit_program:
    mov eax, 1
    xor ebx, ebx
    int 0x80

get_numbers:
    print msg_first, msg_first_len
    call read_number
    mov [num1], eax
    print msg_second, msg_second_len
    call read_number
    mov [num2], eax
    ret

read_char:
    mov eax, 3
    mov ebx, 0
    mov ecx, buf
    mov edx, 16
    int 0x80
    test eax, eax
    jz  exit_program
    mov al, [buf]
    and al, 0xDF
    ret

read_number:
    mov eax, 3
    mov ebx, 0
    mov ecx, buf
```

```
    mov edx, 16
    int 0x80
    test eax, eax
    jz  exit_program
    xor eax, eax
    xor esi, esi
.next_digit:
    movzx ebx, byte [buf + esi]
    cmp bl, '0'
    jb  .done
    cmp bl, '9'
    ja  .done
    imul eax, 10
    sub bl, '0'
    add eax, ebx
    inc esi
    cmp esi, 2
    jb  .next_digit
.done:
    ret

print_number:
    xor esi, esi
    cmp eax, 0
    jge .convert
    neg eax
    mov esi, 1
.convert:
    mov edi, numbuf + 11
    mov ebx, 10
.divloop:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    test eax, eax
    jnz .divloop
    cmp esi, 1
    jne .show
    dec edi
    mov byte [edi], '-'
.show:
```

```
mov ecx, edi
mov edx, numbuf + 11
sub edx, edi
mov eax, 4
mov ebx, 1
int 0x80
ret
```

V. GIT Repository Link

https://github.com/Jabergonia/JBRepo